Milestone 1: Technical Understanding and Research Framing

Christian Percival

Topic

- Seminar topic: Handling execution overruns in hard real-time control systems.
- Main reference paper:
  - M. Caccamo, G. Buttazzo, and L. Sha, "Handling Execution Overruns in Hard Real-Time Control Systems," IEEE Transactions on Computers, 2002.
- Official background reference:
  - G. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed., Springer, 2024.

Problem Statement

- Hard real-time systems must complete tasks before their deadlines.
- A result that is correct but produced too late can still be wrong in a hard real-time system.
- Classical scheduling often uses the worst-case execution time (WCET).
- WCET-based scheduling is safe, but often inefficient.
- Many tasks usually execute much faster than their WCET.
- The problem is how to use the processor efficiently in normal cases while still remaining safe when worst-case execution happens.

What is an Execution Overrun?

- An execution overrun happens when a task executes longer than its expected or reserved execution time.
- It does not mean that the task finishes earlier than WCET.
- Finishing earlier than WCET only means that reserved processor time may be unused.
- In the main paper, the normal computation time is denoted as:

$$c_i^n$$

- The worst-case execution time is denoted as:

$$WCET_i$$

- An overrun occurs when:

$$c_i > c_i^n$$

- The overrun amount is:

$$c_i - c_i^n$$

Motivation from Visual Tracking

- The main paper motivates the problem using visual tracking.
- A vision task may normally search only a small predicted image window.
- If the object is found in this small window, the task finishes quickly.
- If the object is not found, the search area must be expanded.
- Searching a larger area takes more computation time.
- This creates a task with:
  - short normal execution time,
  - occasional long execution time,
  - large WCET.
- Similar situations occur in:
  - robotics,
  - radar tracking,
  - autonomous systems,
  - sensor-based control systems.

Core Technical Idea

- Do not always schedule tasks only according to WCET.
- Use normal execution time to allow higher control frequencies.
- Use WCET information to guarantee safety.
- Use bandwidth reservation to prevent one task from using too much CPU time.
- Use deadline postponement to handle extra execution time.
- Use server-based scheduling to isolate the effect of overruns.

Important Terms

- **Hard real-time task:** a task where missing a deadline is unacceptable.
- **WCET:** worst-case execution time.
- **Normal computation time:** typical runtime of the task.
- **Execution overrun:** task exceeds expected or reserved execution time.
- **EDF:** Earliest Deadline First scheduling.
- **CBS:** Constant Bandwidth Server.
- **CBShd:** CBS extension for hard-deadline overrun handling.
- **PLI:** Performance Loss Index.
- **Temporal isolation:** preventing one task from disturbing other tasks too much.

Mathematical Model

- Each control task can be modeled as:

$$\tau_i(WCET_i, c_i^n, f_i^{min}, \Delta J_i(f_i))$$

- $WCET_i$: worst-case execution time.
- c_i^n: normal computation time.
- f_i^{min}: minimum allowed frequency.
- $\Delta J_i(f_i)$: performance loss index.

Utilization

- Task utilization can be written as:

$$U_i = C_i f_i$$

- Total utilization:

$$U = \sum_{i=1}^{n} U_i$$

- For EDF on one processor, the task set is usually schedulable if:

$$\sum_{i=1}^{n} U_i \leq 1$$

Performance Loss Index

- The paper models performance loss as:

$$\Delta J_i(f_i) = \alpha_i e^{-\beta_i f_i}$$

- f_i: task frequency.
- α_i: magnitude coefficient.
- β_i: decay rate.
- Higher frequency usually reduces performance loss.
- Higher frequency also increases CPU usage.
- The total performance loss is:

$$\Delta J(f_1, \ldots, f_n) = \sum_i w_i \Delta J_i(f_i)$$

CBS and CBShd

- CBS assigns each task a server budget and server period:

$$(Q_s, T_s)$$

- The server bandwidth is:

$$U_s = \frac{Q_s}{T_s}$$

- CBS limits how much processor time a task can consume.

- If the task exhausts its budget, the deadline is postponed.
- CBShd improves CBS for hard-deadline tasks.
- CBShd avoids postponing deadlines too far when the remaining overrun is small.

Assumptions

- Tasks are periodic control tasks.
- WCET is known or can be estimated.
- Normal execution time is known or measured.
- Minimum safe task frequency is known.
- Overruns are bounded by WCET.
- The system uses EDF or a compatible server-based scheduler.
- Remaining computation time can be estimated.

Limitations

- WCET is difficult to estimate on modern processors.
- Normal execution time depends on input data.
- Server management introduces runtime overhead.
- Smaller budgets improve precision but increase scheduling overhead.
- Implementation is easier in a real-time kernel than in a normal desktop operating system.

Open Questions

- How accurately can WCET be measured in a student implementation?
- Should the final paper focus more on CBS/CBShd theory or the implementation example?
- Is a Python simulation acceptable as an implementation demonstration?
- How should the implementation distinguish normal cases, overruns, and worst cases?

- How much mathematical detail should be included in the final 5-page paper?